

Users and Permissions

November 18, 2025

In this assignment you will learn how to add users and control read, write and execute permissions. In the first section of the document, we describe the assignment. The subsequent sections give you relevant information to help you complete the task.

1 Assignment

In figures 1, 2, 3 and 4 you will find an image of the directory tree and the checksums of the files in the tree. Note that figure 3 is the same as 2, just zoomed in so it's easier to read. Your job is to use your command line skills to recreate the directory structure and files shown in these images.

This document is long, because it contains a description of some commands you could use to help you complete the assignment. They are provided as a reference and you can safely skip those sections if, for example, you already know how to use the **adduser** command.

```
root@droplet:~# tree -pug /home
[drwxr-xr-x root      root      ] /home
├── [drwx----- jack      jack      ] jack
│   ├── [drwxr-xr-x jack      jack      ] Documents
│   │   └── [-rw-r--r-- jack      jack      ] homework.txt
│   └── [drwx----- jack      jack      ] MyCode
│       └── [-rwx----- jack      jack      ] script.sh
└── [drwx----- jill      jill      ] jill
    ├── [drwxrwxr-x jill      jill      ] Documents
    │   └── [-rw----- jill      jill      ] diary.txt
    └── [drwxrwxr-x jill      jill      ] MyCode
        └── [-rwxr-xr-x jill      jill      ] program.py

7 directories, 4 files
```

Figure 1: Reproduce this tree

```
root@droplet:/home# find . -type f \( -name *.txt -o -name *.py -o -name *.sh \) | xargs -I {} -n1 md5sum {} ;
438f6ccd43371c586bc2de65b92f9c17 ./jack/MyCode/script.sh
ac77a375cde3de26a411d5a87a3eb5cf ./jack/Documents/homework.txt
2960bd2bd0e3c32ebd87fb477101c104 ./jill/MyCode/program.py
98a75dd1345c4701a9dfc4bea6a406c3 ./jill/Documents/diary.txt
```

Figure 2: Make sure the checksums match.

```
438f6ccd43371c586bc2de65b92f9c17 ./jack/MyCode/script.sh
ac77a375cde3de26a411d5a87a3eb5cf ./jack/Documents/homework.txt
2960bd2bd0e3c32ebd87fb477101c104 ./jill/MyCode/program.py
98a75dd1345c4701a9dfc4bea6a406c3 ./jill/Documents/diary.txt
```

Figure 3: Closeup of checksums so it's easier to read.

```
root@droplet:~# tree -pug /home | md5sum
e2e65bba10436a563602e83ea29a3e4e -
root@droplet:~# tree -pug /home | sha512sum
488ffe65226ff9d8891b1bf0fd7c4a743ed1abfe8222c3e66a5ccd2ad40fd3db8cc0243cf634ceaac9ca399f28fb623
991061d9d53cba8e2ac7c4a911373da69 -
root@droplet:~#
```

Figure 4: It is very difficult for me to read the tree output and verify all the permissions is right. Let's pipe the tree output to md5sum and sha512 sum so I can just verify the checksum! Isn't that a fun use of the checksums you've just learned about this semester?

1.1 The Files

In figures 5, 6, 7 and 8 you will find information about the contents of the files.

```
root@droplet:/home/jack/Documents# cat homework.txt
I love Linux!
root@droplet:/home/jack/Documents# xxd homework.txt
00000000: 4920 6c6f 7665 204c 696e 7578 210a      I love Linux!
root@droplet:/home/jack/Documents#
```

Figure 5: /home/jack/Documents/homework.txt

```
root@droplet:/home/jack/MyCode# cat script.sh
#!/usr/bin/bash

echo "hello world!"
root@droplet:/home/jack/MyCode# xxd script.sh
00000000: 2321 2f75 7372 2f62 696e 2f62 6173 680a  #!/usr/bin/bash.
00000010: 0a65 6368 6f20 2268 656c 6c6f 2077 6f72  .echo "hello wor
00000020: 6c64 2122 0a                                ld!".
```

Figure 6: /home/jack/MyCode/script.sh

```
root@droplet:/home/jill/Documents# ls
diary.txt
root@droplet:/home/jill/Documents# cat diary.txt
today was a good day.
root@droplet:/home/jill/Documents# xxd diary.txt
00000000: 746f 6461 7920 7761 7320 6120 676f 6f64  today was a good
00000010: 2064 6179 2e0a                                day..
```

Figure 7: /home/jill/Documents/diary.txt

```
root@droplet:/home/jill/MyCode# cat program.py
#!/usr/bin/python3

import this
root@droplet:/home/jill/MyCode# xxd program.py
00000000: 2321 2f75 7372 2f62 696e 2f70 7974 686f  #!/usr/bin/pytho
00000010: 6e33 0a0a 696d 706f 7274 2074 6869 730a  n3..import this.
root@droplet:/home/jill/MyCode# |
```

Figure 8: /home/jill/MyCode/program.py

2 Install programs

In this assignment you'll be using the **tree** program. The **ascii** and **xxd** programs will be a bit confusing for you at this point, but I present them so you see them. We'll discuss them in class and you can use them to help you verify that your file are correct. Install them like:

```
1 root@droplet$ apt install tree xxd ascii
```

- **tree** - lets you see your directory structure as a nice graphical tree.
- **xxd** - gives a hex dump of a file so you can see the file in hexadecimal. This is an important command you'll use when inspecting files. We present it in this document, which is likely your first exposure to the xxd command.
- **ascii -x** - will tell you the hex value for every letter, number, punctuation symbol and more in the ascii text encoding. If you look at the output of **ascii -x** you'll see that the hex value **49** is the letter **I** and the hex value **20** is a blank space. Have a look at figure 9

```
root@droplet:~# ascii -x
 00 NUL   10 DLE   20      30 0      40 @      50 P      60 `      70 p
 01 SOH   11 DC1   21 !     31 1      41 A      51 Q      61 a      71 q
 02 STX   12 DC2   22 "     32 2      42 B      52 R      62 b      72 r
 03 ETX   13 DC3   23 #     33 3      43 C      53 S      63 c      73 s
 04 EOT   14 DC4   24 $     34 4      44 D      54 T      64 d      74 t
 05 ENQ   15 NAK   25 %     35 5      45 E      55 U      65 e      75 u
 06 ACK   16 SYN   26 &     36 6      46 F      56 V      66 f      76 v
 07 BEL   17 ETB   27 '     37 7      47 G      57 w      67 g      77 w
 08 BS    18 CAN   28 (     38 8      48 H      58 X      68 h      78 x
 09 HT    19 EM    29 )     39 9      49 I      59 Y      69 i      79 y
 0A LF    1A SUB   2A *     3A :     4A J      5A Z      6A j      7A z
 0B VT    1B ESC   2B +     3B ;     4B K      5B [      6B k      7B {
 0C FF    1C FS    2C ,     3C <    4C L      5C \      6C l      7C |
 0D CR    1D GS    2D -     3D =     4D M      5D ]      6D m      7D }
 0E SO    1E RS    2E .     3E >    4E N      5E ^      6E n      7E ~
 0F SI    1F US    2F /     3F ?     4F O      5F _      6F o      7F DEL

root@droplet:~# xxd /home/jack/Documents/homework.txt
00000000: 4920 6c6f 7665 204c 696e 7578 210a      I love Linux!.
```

Figure 9: See that the first two values in homework.txt are "I" and a SPACE. In the output of **xxd**, you see the first two hex values are "4920", which is exactly the values corresponding to an "I" and a SPACE in **ascii -x**. This will be confusing at first, but with a little time will become very clear and you'll be amazed.

3 Creating the tree

3.1 users

Linux is a multi-user operating system. Our droplets by default only come with one user - root - but you can add more users to your computer.

3.2 adduser

To add a user, you use the **adduser** command. When you add a user you'll be prompted to give a password. After providing the password, you're asked a bunch of questions. You can just hit ENTER repeatedly and not enter values for things like Room Number.

See figure 10. Note that in the beginning there are only directories for jack and jill. When I add the user bruno and **ls /home**, you see that there is now a home directory for bruno. When you run **adduser**, a home directory is added for the user.

```
root@droplet:~# ls /home
jack  jill
root@droplet:~# adduser bruno
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for bruno
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] Y
root@droplet:~# ls /home
bruno jack jill
root@droplet:~#
```

Figure 10: adduser

3.3 groups

Linux users can be binned into groups. This is an important topic, but will be left for a future lecture. If we start discussing groups in this lecture, it might feel like too much material.

3.4 addgroup

You can add groups to your computer with **addgroup**. But, as mentioned above, that is outside of the scope of today's assignment.

3.5 File ownership

Every file in Linux has a user owner and a group owner. See figure 11. The output of **ls -l** shows who is the user owner and who is the group owner of any file or directory.

```
jack@droplet:~$ ls -l
total 8
drwxr-xr-x 2 jack jack 4096 Nov 13 15:09 Documents
drwx----- 2 jack jack 4096 Nov 13 15:11 MyCode
jack@droplet:~$
```

Figure 11: See the output of **ls -l**. The user owner is highlighted in green, and group owner is highlighted in orange.

3.6 chown

To change the owner of a file you can use **chown**. In figure 12 you see how to change the file owner. In figure 13 you see how to change the owner and the group. In figure 14 you see how to change ownership recursively for all of the contents of a directory.

```
root@droplet:~# ls -l
total 0
-rw-r--r-- 1 root root 0 Nov 13 16:09 testfile.txt
root@droplet:~# chown bruno testfile.txt
root@droplet:~# ls -l
total 0
-rw-r--r-- 1 bruno root 0 Nov 13 16:09 testfile.txt
```

Figure 12: See highlighted in yellow that the owner was root. After running the **chown** command, the owner is bruno.

```
root@droplet:~# ls -l
total 0
-rw-r--r-- 1 bruno root 0 Nov 13 16:09 testfile.txt
root@droplet:~# chown melvyn:melvyn testfile.txt
root@droplet:~# ls -l
total 0
-rw-r--r-- 1 melvyn melvyn 0 Nov 13 16:09 testfile.txt
root@droplet:~#
```

Figure 13: See that the user/group of the file was bruno/root. But after *chown*, the user/group are melvyn/melvyn.

```
root@droplet:~# tree -ug mydir
[root      root      ] mydir
└─ [root      root      ] myfile.txt

1 directory, 1 file
root@droplet:~# chown -R melvyn:melvyn mydir
root@droplet:~# tree -ug mydir
[melvyn    melvyn    ] mydir
└─ [melvyn    melvyn    ] myfile.txt
```

Figure 14: Instead of using *ls -l*, I use *tree -ug* so it's easier to see the ownership of a directory and its contents. We use *chown* with the *-R* flag to change the ownership of a directory and its contents.

3.7 Permissions

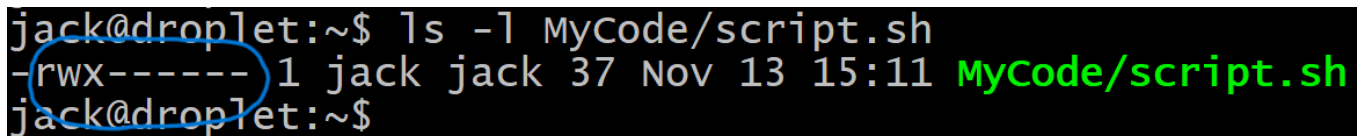
Linux files and directories have permissions. Files can directories can be:

- readable
- writable
- executable

These permissions are specified in three ways:

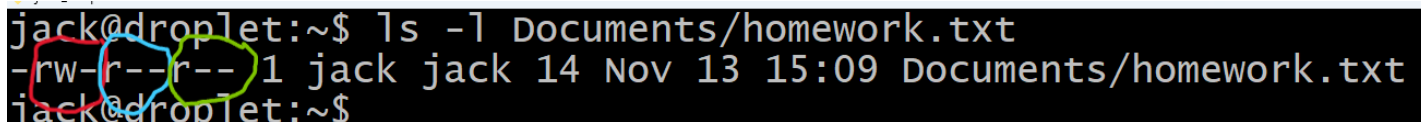
- owner
- group
- everyone else

So files and directories have 9 permission fields. They have readable, writable and executable for each of owner, group and everyone else. See figures 15 and 16 to see two examples of permissions. Permissions are show with 9 characters.



```
jack@droplet:~$ ls -l MyCode/script.sh
-rwx----- 1 jack jack 37 Nov 13 15:11 MyCode/script.sh
jack@droplet:~$
```

Figure 15: See the circled area. 9 characters indicate read, write an execute for each of owner, group, and everyone else



```
jack@droplet:~$ ls -l Documents/homework.txt
-rw-r--r-- 1 jack jack 14 Nov 13 15:09 Documents/homework.txt
jack@droplet:~$
```

Figure 16: The red circle indicates permissions for owner. Blue indicates group. Green indicates everyone else

3.8 chmod

TODO

4 The Checksums

TODO